

. Matteo Calvanico

☎ +39-347-385-2782 ✉ matteo.calvanico03@gmail.com  [Matteo Calvanico](#)  [MatteoCalvanico](#)

SKILLS

- **Languages:** Java, C# (.NET 8.0), C++, C89, Python, Kotlin, Javascript, Typescript, SQL, GDScript, LaTeX.
- **Frameworks & Libraries:** Vue, React, Jest, Zod, Mongoose, Serverless Application Model (SAM), Entity Framework, .NET, Bootstrap, TensorFlow, Scikit Learn, PyTorch, CUDA, OpenCV, Pandas, Numpy, Gradle.
- **Tools:** Visual Studio, VS Code, Andorid Studio, Arduino, Google Cloud Console, AWS, Git, GitHub, Godot, Docker, Jenkins, Dockerhub.

EDUCATION

- **"Alma Mater Studiorum" - University of Bologna** Cesena (FC), Italy
Bachelor's Degree in Computer System Technologies 2022 – 2025
 - **Skills:** Object-Oriented Programming (Java, C#), Mobile Systems Programming (Kotlin, Android Studio, Room, Gradle, Retrofit), Databases (SQL, MySQL), Web Systems Engineering (JavaScript, TypeScript, CSS, SCSS/SASS, NodeJS, Vue.js, Bootstrap), Basics of Neural Networks, Machine Learning, Image Processing (OpenCV, CUDA, Python, PyTorch, TensorFlow, Scikit Learn, Pandas), Embedded Systems and Internet of Things (Arduino, ESP32, Raspberry Pi, MQTT), Development Platforms for Automation and Virtualization (Docker, Vagrant, Kubernetes, Active Directory), Network Systems, Development in Asp.NET
- **I.T.T. "Blaise Pascal"** Cesena (FC), Italy
Diploma in Computer Science September 2017 – June 2022
 - **Description:** I have completed the high school with a specialization in Computer Science and Telecommunication Systems. I have studied the basics of programming, networking, electronics, and much more.

WORK EXPERIENCE

- **DMA - CX Company for B2B** Cesena (FC), Italy
Intern Software Engineer Aprile 2025 – Agosto 2025
 - **Technologies used:** AWS (Cloudformation, Lambda, API Gateway, SQS, Timestream, IoT Core, EC2), React, Grafana, Docker, GitHub Actions, MongoDB Atlas
 - **Description:** The collaboration with *DMA* led to the development of a *Dashboard*, where customer service operators of companies relying on DMA for their software solutions can monitor the current and past status of various devices used by different consumers, in order to provide targeted and functional support in case of failures or problems of any kind. The creation of the dashboard was not straightforward; before beginning the design and development of the product, it was necessary to acquire new knowledge and consolidate existing one across various technologies. The process was step-by-step, involving the development of smaller projects with progressively increasing difficulty, which collectively gathered the various skills necessary for dashboard development.
- **Maggioli S.P.A.** Santarcangelo di Romagna (FC), Italy
Intern Software Engineer November 2024 – March 2025
 - **Technologies used:** Angular, Java, Spring, Postman, Docker, MongoDB, GitLab
 - **Description:** Italian company that provides software solutions for public administration. Working as a Full-Stack programmer I learned to move in an already established code base and to work in a large team, learning how to deal with the daily problems that can arise at any time in a large and dynamic company.

PROJECTS

- **ECC Dashboard:** Cloud-based application for customer support.
 - **AWS:** the application, in its back-end part, uses various technologies from the AWS suite to function. Among these: **IoT Core** to retrieve data from field devices, **Timestream** to save incoming data, **Lambda** to perform operations on data sources and more.

- **Grafana and React:** the front-end is divided into two parts. The dashboard uses Grafana to display data sent by devices, with the ability to filter and view past data. The portal with information about devices, users, and alarms is displayed through an application developed in React.
 - **Automated Deployment:** the entire cloud infrastructure and front-end application are deployed through an automated process created using GitHub Actions.
 - **Thesis:** the design and development of the entire application were documented in my bachelor's thesis.
- **Game Vault:** Android app for tracking your game collection, with rating and search capabilities.
 - **Play Store:** the application was tested by 20 people and approved by Google, now available for download directly from the Google Play Store
 - **API:** the search is done through public APIs, which provide all the necessary information about the game to be displayed to the user; the returned JSON files were managed using Moshi and Retrofit.
 - **DB:** to save information persistently, Room was used.
- **GetEat:** Web app for restaurant management, with the ability for users to purchase products and for the admin to manage them.
 - **Multi-technology:** the web app was created using the Vue.js framework, utilizing TypeScript and Scss with the use of Express.js to handle calls to the relational DB created using SQL.
 - **Modularity:** the application was divided first into Front-end/Back-end and then further subdivided into modular files within each part.
 - **Authentication:** the application includes a registration and login part with permission control, thus hiding the admin part from basic users
 - **Accessibility:** the goal was not only to have a functional application but also a responsive and accessible web app.
- **Virtual Casino:** Desktop application in Java simulating some casino games, such as Blackjack, Roulette, and dice.
 - **MVC:** to make the project less complex and facilitate easy modification and testability, the MVC Pattern was used, separating the presentation part from the data part, ensuring communication between the two through the controller
 - **Collaboration:** the project was developed together with two other colleagues, giving us a first taste of teamwork, leading to an equitable division of work, regular virtual calls to discuss the tasks, and helping each other in case of difficulties.
 - **Test-Driven Development:** a crucial part of the project was to perform periodic and automated tests to ensure that each part works correctly, made possible by JUnit.
 - **Gradle:** to automate the compilation and execution of tests, we used Gradle, which was also very useful for dependency management.
- **MultiClass Classification:** Neural network for creating and testing a machine learning model for multi-class classification of different types of fruit. The network is a classic CNN with a non-linear activation function (ReLU) and an SGD optimizer.
 - **CONDA:** Anaconda was essential for creating an environment with everything needed for the development and testing of the network.
 - **Multi-library:** To create, train, and test the network, several libraries such as Pytorch and OpenCV were necessary, which allowed modifying the dataset images to adjust certain aspects.
 - **Kaggle:** to find a dataset that was perfect for my needs, I conducted several searches and ended up on Kaggle, which also allowed me to use its API for downloading images.
- **DoomClone:** Video game developed in C++ similar to old classics like Doom or Wolfenstein 3D.
 - **Basics of computer graphics:** to create a 3D effect through a 2D map, I ventured into the technique of Raycasting, where various mathematical formulas are used to display the content of a vector (which contains the map) on the screen; this is done for each vertical line of the screen.
 - **SDL2:** to display the render on the screen and handle various events, the SDL2 library was necessary.
 - **Event Handle:** To handle various cases such as enemy elimination, movement, or door opening, an event handler was necessary in the main game loop, also using SDL2.
- **ProtoCRPG:** Prototype of a CRPG developed in Godot 4.3 as a starting point for similar games.
 - **Project based learning:** the goal of the project is to learn not only the technology itself (Godot Engine and GDScript) but also how to work on a real game, with all the problems of the case.
 - **Passion:** The ultimate goal is to have a usual base to start working, in the future, on my own isometric role-playing game, following in the footsteps of old classics such as Fallout and Baldur's Gate.
 - **Why Godot?:** the choice of this Game Engine is not accidental, the decision to choose an open source tool is due to the possibility of being free to experiment and also study how these technologies work behind the scenes.